BTK AKADEMİ

İLERİ SEVİYE PYTHON PROGRAMLAMA DİLİ

Veri Akışı (Iterators and Generators)

EBRU AYDOĞAN

Iterable ve Iterator – Döngülerin Arka Planı

- "For" Döngüsü Arka Planda Ne Yapıyor?
- **Soru:** for item in [1, 2, 3]: dediğimizde Python aslında ne yapıyor?
- **Gerçek:** Python, listenin (iterable) üzerine doğrudan atlamaz. Önce listenin __iter__ metodunu çağırıp bir **Iterator** nesnesi ister. Sonra da bu iterator bitene kadar sürekli __next__ metodunu çağırır.
- Analoji:
  - **Iterable:** Kitabın kendisi (üzerinde gezinebilirsiniz).
  - **Iterator:** Kitap ayracı (o an hangi sayfada olduğunuzu tutan ve bir sonraki sayfaya geçmenizi sağlayan araç).

BTK AKADEMİ

Iterable ve Iterator – Döngülerin Arka Planı

```
liste = [1, 2, 3]

# Liste bir Iterable'dır. Ama iterator değildir.
# iter() fonksiyonu, listenin 'iterator' versiyonunu bize verir.
iterator = iter(liste)

# Artık 'next()' diyerek tek tek ilerleyebiliriz
print(next(iterator)) # 1
print(next(iterator)) # 2
print(next(iterator)) # 3
# print(next(iterator)) # HATA! StopIteration (Iterator bitti)
```

BTK AKADEMİ

Custom Iterator Yazmak

- Artık hazır listelerle yetinmiyoruz, kendi veri yapılarımızı "döngüye girebilir" (loopable) hale getiriyoruz.
- **Kural:** Bir sınıfın iterator olması için iki zorunlu metodu vardır:
  - __iter__: Nesnenin kendisini (iterator) döndürür.
  - __next__: Bir sonraki elemanı döndürür veya StopIteration fırlatır.

BTK AKADEMİ

Custom Iterator Yazmak

```python
class MyNumbers:
    def __init__(self, start, stop):
        self.start = start
        self.stop = stop

    def __iter__(self):
        # Nesneyi iterator'a dönüştürdük
        return self

    def __next__(self):
        if self.start <= self.stop:
            x = self.start
            self.start += 1
            return x
        else:
            # Döngüyü bitiren sihirli komut
            raise StopIteration

# Kullanım:
for x in MyNumbers(10, 15):
    print(x)
```

BTK AKADEMİ

Custom Iterator Yazmak

- Kritik Notlar:
  - **Döngü vs next():** for x in MyNumbers(10, 15): dediğimizde Python arka planda StopIteration hatasını "yakalar" (catch eder) ve döngüyü sessizce bitirir. Biz next() ile manuel çağırırsak hatayı görürüz ama for döngüsü bunu yönetir.
  - **Neden Sınıf Yazıyoruz?:** Çünkü bazen verimiz bir listede durmaz; mesela dosyanın içinde olabilir, veritabanından satır satır geliyor olabilir veya sonsuz bir sayı dizisi olabilir. RAM'i doldurmadan, sadece o an ihtiyacımız olan elemanı (iterator) üretmek için bunu yapıyoruz.
  - **__iter__ neden self döner?:** Çünkü biz bu sınıfın kendisini bir iterator olarak kullanıyoruz, o yüzden self (nesnenin kendisi) diyoruz.

Generators (Bellek Dostu Python)

- Bellek Yiyen Listeler vs. Hafif Generator'lar
- **Problem:** [i for i in range(1000000)] dediğinizde Python bu 1 milyon sayıyı RAM'de tutar. Bellek doldu, bilgisayar yavaşladı.
- **Çözüm: Generator.** "Ben sayıları önceden hazırlayıp RAM'e koymuyorum; sen ne zaman 'bana sayı ver' (next) dersen, o an üretiyorum."
- Analoji:
  - **Listeler:** Buzdolabında önceden pişmiş 1000 tane yemek (RAM'i doldurur).
  - **Generator:** Mutfağa girince o an pişirilen taze yemek (Sadece ihtiyaç anında üretilir, RAM dostudur).

BTK AKADEMİ

Generators (Bellek Dostu Python)

- yield Anahtar Kelimesi
- **Mantık:** return fonksiyonu tamamen bitirip bellekten siler. yield ise fonksiyonu "dondurur" (pause), değerini gönderir, dışarıdan tekrar çağrıldığında kaldığı yerden devam eder.

```
def square_generator(n):
    for i in range(n):
        yield i ** 2  # 'return' değil 'yield'!

gen = square_generator(5)
# gen bir 'generator' nesnesidir, liste değil!
for x in gen:
    print(x) # 0, 1, 4, 9, 16 (Anlık üretildi)
```

BTK AKADEMİ

Generators (Bellek Dostu Python)

- Generator Expressions
- List comprehension [...] ile yapılıyorsa, Generator expression (...) ile yapılır.

```
# Liste: [0, 1, 4, 9, 16] -> Bellekte kaplar
my_list = [i**2 for i in range(5)]

# Generator: <generator object ...> -> Bellekte kaplamaz, iterable'dır
my_gen = (i**2 for i in range(5))
```

BTK AKADEMİ

Uygulama

1. (1 - ∞) aralığındaki her sayının karesini getiren fonksiyon.
2. Fibonacci serisini hem normal fonksiyon hem de generator fonksiyon ile oluşturun.
3. Performans testlerini yapın.